

제 2 주:

배열 가방 – 기본 기능

동전가방



강 지 훈

jhkang@cnu.ac.kr

충남대학교 컴퓨터공학과



[제 2 주 실습]

ArrayBag 을 사용하는 동전 가방



□ ArrayBag과 관련된 필요한 기능

- 초기화
 - 코인 값들의 초기화
- 코인 입력
- 코인 수 세기
 - 코인을 정상적으로 저장할 때마다 +1
- 코인 삭제
- 전체 코인 수
- 코인이 가방에 있는지 확인
- 가방 안에 있는 특정 코인의 개수

이 과제에서 필요한 객체는?

- ApplicationController

- AppView

- Model

 - ArrayBag <E>

 - Coin

□ 입출력

■ 메뉴 입력에 따른 할 일

- 처음 가방에 들어갈 최대 가방 사이즈를 입력한다.
- 메뉴화면을 보여준다.
- 9가 입력 되면 "<9가 입력되어 종료합니다>" 라는 메시지를 내보내고 프로그램을 종료한다
- 1 (add) 이 입력되면, 동전의 액수를 입력 받아 동전 가방에 동전을 넣는 일을 실행한다
- 2 (remove) 가 입력되면, 동전의 액수를 입력 받아 동전 가방에서 동일한 값을 갖는 동전 하나를 삭제한다.
- 3 (search) 이 입력 되면, 동전의 액수를 입력 받아 동전 가방에 동일한 값을 갖는 동전이 존재하는지 검색하여 그 여부를 출력한다.
- 4 (frequency) 가 입력되면, 코인의 액수를 입력 받아 동전 가방에 동일한 값을 갖는 동전의 개수를 세어 출력한다.

■ 통계 출력

- 프로그램 종료 시점에, 총 동전의 개수, 동전 중에서 가장 큰 값, 모든 동전들의 값의 합을 출력한다.

□ 출력의 예 [1]

<< 동전 가방 프로그램을 시작합니다 >>

가방에 들어갈 총 동전의 개수를 입력하시오: 6

? 수행하려고 하는 메뉴 번호를 선택하시오

(add: 1, remove: 2, search: 3, frequency: 4, exit: 9) : 1

? 동전 값을 입력하시오: 5

- 주어진 값을 갖는 동전을 가방에 성공적으로 넣었습니다.

? 수행하려고 하는 메뉴 번호를 선택하시오

(add: 1, remove: 2, search: 3, frequency: 4, exit: 9) : 1

? 동전 값을 입력하시오: 10

- 주어진 값을 갖는 동전을 가방에 성공적으로 넣었습니다.

? 수행하려고 하는 메뉴 번호를 선택하시오

(add: 1, remove: 2, search: 3, frequency: 4, exit: 9) : 1

? 동전 값을 입력하시오: 20

- 주어진 값을 갖는 동전을 가방에 성공적으로 넣었습니다.

? 수행하려고 하는 메뉴 번호를 선택하시오

(add: 1, remove: 2, search: 3, frequency: 4, exit: 9) : 1

? 동전 값을 입력하시오: 5

- 주어진 값을 갖는 동전을 가방에 성공적으로 넣었습니다.

? 수행하려고 하는 메뉴 번호를 선택하시오

(add: 1, remove: 2, search: 3, frequency: 4, exit: 9) : 1

? 동전 값을 입력하시오: 30

- 주어진 값을 갖는 동전을 가방에 성공적으로 넣었습니다.

? 수행하려고 하는 메뉴 번호를 선택하시오

(add: 1, remove: 2, search: 3, frequency: 4, exit: 9) : 1

? 동전 값을 입력하시오: 5

- 주어진 값을 갖는 동전을 가방에 성공적으로 넣었습니다.

? 수행하려고 하는 메뉴 번호를 선택하시오

(add: 1, remove: 2, search: 3, frequency: 4, exit: 9) : 1

? 동전 값을 입력하시오: 5

- 가방이 꽉 차서 주어진 값을 갖는 동전을 가방에 넣는데 실패하였습니다.

□ 출력의 예 [2]

? 수행하려고 하는 메뉴 번호를 선택하시오
 (add: 1, remove: 2, search: 3, frequency: 4, exit: 9) : 2
 ? 동전 값을 입력하시오: 50
 - 주어진 값을 갖는 동전은 가방 안에 존재하지 않습니다.

? 수행하려고 하는 메뉴 번호를 선택하시오
 (add: 1, remove: 2, search: 3, frequency: 4, exit: 9) : 2
 ? 동전 값을 입력하시오: 5
 - 주어진 값의 동전 하나가 가방에서 정상적으로 삭제되었습니다.

? 수행하려고 하는 메뉴 번호를 선택하시오
 (add: 1, remove: 2, search: 3, frequency: 4, exit: 9) : 3
 ? 동전 값을 입력하시오: 20
 - 주어진 값을 갖는 동전이 가방 안에 존재합니다.

? 수행하려고 하는 메뉴 번호를 선택하시오
 (add: 1, remove: 2, search: 3, frequency: 4, exit: 9) : 3
 ? 동전 값을 입력하시오: 70
 - 주어진 값을 갖는 동전은 가방 안에 존재하지 않습니다.

? 수행하려고 하는 메뉴 번호를 선택하시오
 (add: 1, remove: 2, search: 3, frequency: 4, exit: 9) : 4
 ? 동전 값을 입력하시오: 5
 - 주어진 값을 갖는 동전의 개수는 3 개 입니다.

? 수행하려고 하는 메뉴 번호를 선택하시오
 (add: 1, remove: 2, search: 3, frequency: 4, exit: 9) : 4
 ? 동전 값을 입력하시오: 80
 - 주어진 값을 갖는 동전의 개수는 0 개 입니다.

? 수행하려고 하는 메뉴 번호를 선택하시오
 (add: 1, remove: 2, search: 3, frequency: 4, exit: 9) : 9
 - 가방에 대한 수행을 종료합니다.

출 동전의 개수: 5 개
 동전 중에서 가장 큰 값: 30
 모든 동전들의 값의 합: 70

<< 동전 가방 프로그램을 종료합니다 >>

main()



□ main()을 위한 class

```
public class DS1_02_학번_이름 {  
    public static void main (String[] args)  
    {  
        ApplicationController appController = new ApplicationController() ;  
        // ApplicationController 가 실질적인 main class 이다.  
        appController.run() ;  
        // 여기 main()에서는 앱 실행이 시작되도록 해주는 일이 전부이다.  
    }  
}
```

Class “AppController”의 구현



□ Class "AppController" 의 구현 [1]

```
public class AppController {  
    // 비공개 인스턴스 변수들  
    private AppView        _appView ;  
    private ArrayBag<Coin>  _coinBag ;  
  
    // 생성자  
    public AppController()  
    {  
        this._appView = new AppView() ;  
    }  
  
    // 비공개함수의 구현  
    .....  
  
    // 공개함수의 구현  
    .....  
} // End of class "AppController"
```

□ ApplicationController: 단계 1

```
public class ApplicationController {
```

```
..... // 생략
```

```
// 공개함수의 구현
```

```
public void run() {
```

```
    this._appView.outputLine("<<< 동전 가방 프로그램을 시작합니다 >>>");
```

```
    int menuNumber = this._appView.inputMenuNumber();
```

```
    while ( menuNumber != MENU_EXIT ) {
```

```
        // 단계 2 에서 채워야 할 곳
```

```
        menuNumber = this._appView.inputMenuNumber();
```

```
    }
```

```
    this._appView.outputLine("<<< 동전 가방 프로그램을 종료합니다 >>>");
```

```
}
```

```
} // End of class "AppController"
```

□ ApplicationController: 단계 2

```

public class ApplicationController {
    // 상수
    private static final int MENU_ADD = 1 ;
    private static final int MENU_REMOVE = 2 ;
    private static final int MENU_SEARCH = 3 ;
    private static final int MENU_FREQUENCY = 4 ;
    private static final int MENU_EXIT = 9 ;

    ..... // 생략

    // 공개함수의 구현
    public void run() {
        this._appView.outputLine("<<< 동전 가방 프로그램을 시작합니다 >>>") ;

        // 단계 3 에서 채워야 할 곳

        int menuNumber = this._appView.inputMenuNumber() ;
        while ( menuNumber != MENU_EXIT ) {
            switch ( menuNumber ) {
                case MENU_ADD:
                    // 단계 3 에서 채워야 할 곳
                    break;
                case MENU_REMOVE:
                    // 단계 3 에서 채워야 할 곳
                    break;
                case MENU_SEARCH:
                    // 단계 3 에서 채워야 할 곳
                    break;
                case MENU_FREQUENCY:
                    // 단계 3 에서 채워야 할 곳
                    break;
                default:
                    // 단계 3 에서 채워야 할 곳
            }
            menuNumber = this._appView.inputMenuNumber() ;
        }

        this._appView.outputLine("<<< 동전 가방 프로그램을 종료합니다 >>>") ;
    }
}

```



□ ApplicationController: 단계 3 [1]

```

public class ApplicationController {
    // 상수
    private static final int MENU_ADD = 1 ;
    ..... // 생략

    // 비공개 인스턴스 변수
    ..... // 생략

    // 생성자
    ..... // 생략

    // 비공개 함수의 구현
    private void addCoin () {
        // 단계 4 에서 채워야 할 곳
    }

    private void removeCoin () {
        // 단계 4 에서 채워야 할 곳
    }

    private void searchForCoin () {
        // 단계 4 에서 채워야 할 곳
    }
    private void frequencyOfCoin () {
        // 단계 4 에서 채워야 할 곳
    }

    private void undefinedMenuNumber (int menuNumber) {
        // 단계 4 에서 채워야 할 곳
    }
    private void showStatistics () {
        // 단계 4 에서 채워야 할 곳
    }

    // Public Method
    public void run() {
        this._appView.outputLine("<<< 동전 가방 프로그램을 시작합니다 >>>");

        int capacityOfCoinBag = this._appView.inputCapacityOfCoinBag () ;
        this._coinBag = new ArrayBag<Coin> (capacityOfCoinBag) ;
    }
}

```



□ ApplicationController: 단계 3 [2]

```

int menuNumber = this._appView.inputMenuNumber() ;
while ( menuNumber != MENU_EXIT ) {
    switch (menuNumber) {
        case MENU_ADD:
            this.addCoin () ;
            break;
        case MENU_REMOVE:
            this.removeCoin () ;
            break;
        case MENU_FREQUENCY:
            this.searchForCoin () ;
            break;
        case MENU_SUM_OF_COINS:
            this.frequencyOfCoin () ;
            break;
        default:
            this.undefinedMenuNumber (menuNumber) ;
    }
    menuNumber = this._appView.inputMenuNumber() ;
}
this.showStatistics();
this._appView.outputLine("<<< 동전 가방 프로그램을 종료합니다 >>>") ;
}
} // End of class "AppController"

```


□ ApplicationController: 단계 4 [1]

```
public class ApplicationController {
    ..... // 생략

    // 비공개 함수의 구현
    private void addCoin () {
        int coinValue = this._appView.inputCoinValue () ;
        if ( this._coinBag.add (new Coin(coinValue)) ) {
            this._appView.outputLine("- 주어진 값의 동전을 가방에 성공적으로 넣었습니다.");
        }
        else {
            this._appView.outputLine("- 주어진 값의 동전을 가방에 넣는데 실패하였습니다.");
        }
    }

    private void removeCoin () {
        int coinValue = this._appView.inputCoinValue () ;
        if ( ! this._coinBag.remove (new Coin(coinValue)) ) {
            this._appView.outputLine("- 주어진 값을 갖는 동전은 가방 안에 존재하지 않습니다.");
        }
        else {
            this._appView.outputLine ("- 주어진 값을 갖는 동전 하나가 가방에서 정상적으로 삭제되었습니다.");
        }
    }

    private void searchForCoin () {
        int coinValue = this._appView.inputCoinValue () ;
        if ( this._coinBag.contains (new Coin(coinValue)) ) {
            this._appView.outputLine ("- 주어진 값을 갖는 동전이 가방 안에 존재합니다.");
        }
        else {
            this._appView.outputLine("- 주어진 값을 갖는 동전은 가방 안에 존재하지 않습니다.");
        }
    }

    private void frequencyOfCoin () {
        int coinValue = this._appView.inputCoinValue () ;
        int frequency = this._coinBag.frequencyOf (new Coin(coinValue)) ;
        this._appView.outputLine ("- 주어진 값을 갖는 동전의 개수는 " + frequency + " 개 입니다.");
    }

    private void undefinedMenuNumber (int aMenuNumber) {
        this._appView.outputLine ("- 선택된 메뉴 번호 " + aMenuNumber + " 는 잘못된 번호입니다.");
    }
}
```

□ ApplicationController: 단계 4 [2]

```
public class ApplicationController {
    ..... // 생략

    // 비공개 함수의 구현
    private void addCoin () {.....}
    private void removeCoin () {.....}
    private void searchForCoin () {.....}
    private void frequencyOfCoin () {.....}
    private void undefinedMenuNumber (int aMenuNumber) {.....}

    private void sumOfCoinValues() {
        int sum = 0;
        for (int i = 0; i < this._coinBag.size() ; i++) {
            sum += this._coinBag().elementAt(i).value() ;
        }
        return sum ;
    }
    private int maxCoinValue() {
        int maxValue = 0 ;
        for (int i = 0; i < this._coinBag.size() ; i++) {
            if ( maxValue < this._coinBag().elementAt(i).value() ) {
                maxValue = this._coinBag().elementAt(i).value() ;
            }
        }
        return maxValue ;
    }

    private void showStatistics() {
        this._appView.outputLine ("총 동전 개수: " + this._coinBag.size()) ;
        this._appView.outputLine ("동전 중 가장 큰 값: " + this.maxCoinValue()) ;
        this._appView.outputLine ("동전 값의 합: " + this.sumOfCoinValues()) ;
    }
}
```

□ ApplicationController: 단계 4 [3]

// 공개함수의 구현

```
public void run() {
    this._appView.outputLine("<<< 동전 가방 프로그램을 시작합니다 >>>");

    int capacityOfCoinBag = this._appView.inputCapacityOfCoinBag ();
    this._coinBag = new ArrayBag<Coin> (capacityOfCoinBag);

    int coinValue ;
    int menuNumber = this._appView.inputMenuNumber() ;
    while ( menuNumber != MENU_EXIT ) {
        switch (menuNumber) {
            case MENU_ADD:
                this.addCoin () ;
                break;
            case MENU_REMOVE:
                this.removeCoin () ;
                break;
            case MENU_SEARCH:
                this.searchForCoin () ;
                break;
            case MENU_FREQUENCCY:
                this.frequencyOfCoin () ;
                break;
            default:
                this.undefinedMenuNumber (menuNumber) ;
                break;
        }
        menuNumber = this._appView.inputMenuNumber() ;
    }
    this._appView.outputLine("<<< 동전 가방 프로그램을 종료합니다 >>>");
}
} // End of class "AppController"
```

Class "AppView"



```
public class AppView {  
    // 비공개 상수/변수들  
    private Scanner    _scanner ;  
  
    // 생성자  
    public AppView ()  
    {  
        this._scanner = new Scanner(System.in) ;  
    }  
  
    // 공개함수의 구현  
    .....  
}
```

□ Class "AppView" 의 구현

```
public class AppView {  
    // 비공개 상수/변수들  
    ....  
    // 생성자  
    .....  
  
    public int inputInt () { ... }  
    public int inputCapacityOfCoinBag() { ... }  
    public int inputMenuNumber () { ... }  
    public int inputCoinValue () { ... }  
  
    public void output (String aString) { .... }  
    public void outputLine (String aString) { .... }
```

Class "Coin"



□ Coin: 생성자, 공개함수

■ 생성자

- `public Coin ( ) ;`
 - ◆ Coin 객체를 생성한다. 동전의 값은 모르는 상태이다.
- `public Coin (int givenValue) ;`
 - ◆ 전달받은 값을 갖는 새로운 Coin 객체를 생성한다.

■ 공개함수

- `public int value ( ) ;`
 - ◆ 코인의 금액을 얻는다.
- `public void setValue (int newValue) ;`
 - ◆ 전달받은 newValue를 현재 코인의 금액으로 설정한다.
- `public boolean equals (Object aCoin) ;`
 - ◆ 현재 코인의 금액이 주어진 aCoin 의 금액과 같은 지 확인한다

Class “Coin” 의 구현



□ Coin: 인스턴스 변수

```
public class Coin {  
    private int _value; // 금액
```

□ Coin: 생성자, 공개함수

■ 생성자

- public Coin ()
- public Coin (int givenValue)
 - ◆ this._value 를 givenValue 값으로 설정한다.

■ 공개 함수

- public int value ()
 - ◆ this._value 값을 돌려준다.
- public void setValue (int newValue)
 - ◆ this._value 를 newValue 값으로 설정한다.
- public boolean equals (Object aCoin)
 - ◆ aCoin 의 실제 class 가 Coin 인지 확인한다. Coin 이 아니면 false.
 - ◆ aCoin 의 class 가 Coin 이면, this._value 와 aCoin 의 _value 를 비교한다.
 - 동일한 값을 가지고 있으면 true, 아니면 false 를 얻는다.

@Override

```
public boolean equals (Object aCoin) {
    if ( aCoin.getClass() != Coin.class ) {
        return false;
    }
    else { // aCoin 의 class 를 안전하게 Coin class 로 형 변환 가능.
        return ( this.value() == ((Coin)Object).value() );
    }
}
```

Class "ArrayBag<E>"



□ ArrayBag<E>: 생성자, 공개함수[1]

■ 생성자

- `public ArrayBag () ;`
- `public ArrayBag (int givenCapacity) ;`

■ 공개함수[1]

- `public int size () ;`
- `public boolean isEmpty () ;`
 - ◆ 배열이 비어 있는지 확인한다.
- `public boolean isFull () ;`
 - ◆ 배열이 가득 차 있는지 확인한다.

□ ArrayBag: 공개함수[2]

■ 공개함수[2]

- `public boolean doesContain(E anElement) ;`
 - ◆ 가방 안에 주어진 `anElement` 와 동일한 원소가 존재하는지 확인한다.
- `public int frequencyOf (E anElement) ;`
 - ◆ 가방 안에 주어진 원소 `anElement` 가 몇 개 있는지 돌려준다.
- ~~● `public int sumOfElementValues () ;`~~
 - ~~◆ 전체 동전의 값의 합계를 돌려준다.~~

□ ArrayBag: 공개함수[3]

■ 공개함수[3]

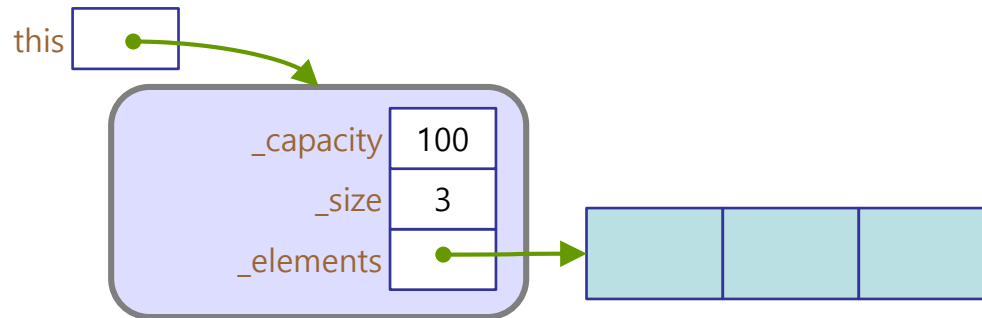
- `public boolean add (E anElement) ;`
 - ◆ 가방에 `anElement` 를 추가한다.
- `public boolean remove (E anElement) ;`
 - ◆ 가방에서 주어진 `anElement` 를 찾아 삭제한다.
- `public void clear() ;`
 - ◆ 가방을 초기화 한다.
- `E elementAt (int anOrder)`
 - ◆ 주어진 순서 `anOrder` 에 있는 원소를 돌려준다.
 - ◆ 동전의 합이나 최대 값이 필요할 경우, 이 함수를 사용하여 모든 동전 객체를 얻어내어 처리한다.

Class “ArrayBag” 의 구현



□ ArrayBag 객체의 구조 – 비공개 인스턴스 변수

```
public class ArrayBag<E> {
    // 비공개 인스턴스 변수
    private static final int DEFAULT_CAPACITY = 100 ;
    private int _capacity ;
    private int _size ;
    private E _elements[] ; // ArrayBag의 원소들을 담을 java 배열
```



□ ArrayBag<E>: 생성자, 공개함수[1]

■ 생성자

- public ArrayBag ()
 - ◆ _capacity 를 DEFAULT_CAPACITY 로 초기화
 - ◆ _elements 를 _capacity 만큼 생성
 - ◆ _size를 0 으로 초기화
- public ArrayBag (int givenCapacity)
 - ◆ _capacity 를 givenCapacity 로 초기화
 - ◆ _elements 를 _maxSize 만큼 생성
 - ◆ _size 를 0 으로 초기화

■ 공개함수[1]

- public int size ()
 - ◆ _size 를 반환
- public boolean isEmpty ()
 - ◆ _size 가 0 인지 확인한다.
- public boolean isFull ()
 - ◆ _size 가 _capacity 와 같은지 확인한다.

□ ArrayBag: 공개함수[2]

■ 공개함수[2]

- `public boolean doesContain (E anElement)`
 - ◆ 찾으면 확인 할 수 있는 `found` 변수를 선언 후 `false`로 초기화
 - ◆ `_size`만큼 각 `_elements`를 확인하여
 - ◆ 만약 같은 값(`equals`)이면 `found`를 `true`로 변경
 - ◆ `_elements`의 값을 모두 확인 하였거나 `found`가 `true`이면 찾는 것을 종료
 - ◆ `found` 값을 반환
- `public int frequencyOf (E anElement)`
 - ◆ 개수를 저장 할 `frequencyCount`를 선언하여 초기화
 - ◆ `_elements` 를 모두 확인하며 `anElement`의 `value`와 비교
 - ◆ 값이 `value` 와 같으면 `frequencyCount` 를 1 증가
 - ◆ `frequencyCount`를 반환

□ ArrayBag: 공개함수[3]

■ 공개함수[3]

- `public boolean add (E anElement)`
 - ◆ 가득 차 있는 경우(`isFull()`) `false`를 반환
 - ◆ `_elements[]` 배열에 `anElement` 값을 저장
 - ◆ `_size` 를 증가
- `public boolean remove (E anElement)`
 - ◆ `_elements[]` 가 비어있는 경우 `false`를 돌려준다
 - ◆ `_elements[]` 를 확인하며 `anElement` 와 같은 값이 있는 위치를 찾음
 - ◆ 위치를 찾지 못하면 `false`를 반환
 - ◆ 원소의 위치를 찾으면 원소를 삭제 하고, 삭제 된 원소 이후의 모든 배열의 원소들을 앞으로 이동
 - ◆ 마지막 위치의 값을 `null` 로 초기화 한다.
 - ◆ 이동이 완료 되면 `true`를 돌려준다.
- `Public void clear()`
 - ◆ `_size` 를 초기화한다.

□ ArrayBag: 공개함수[4]

■ 공개함수[4]

- E elementAt (int anOrder)
 - ◆ 주어진 순서 anOrder 에 있는 원소를 돌려준다.

```
public E elementAt (int anOrder) {  
    if ( (0 <= order) && (order < this.size()) ) {  
        return this._elements [anOrder] ;  
    }  
    else {  
        return null ;  
    }  
}
```

[제 2 주] 실습 끝



